Adrian Hansen, Marko Misic

Subtask 1: Naive Implementation & Deadlock Analysis

Observation

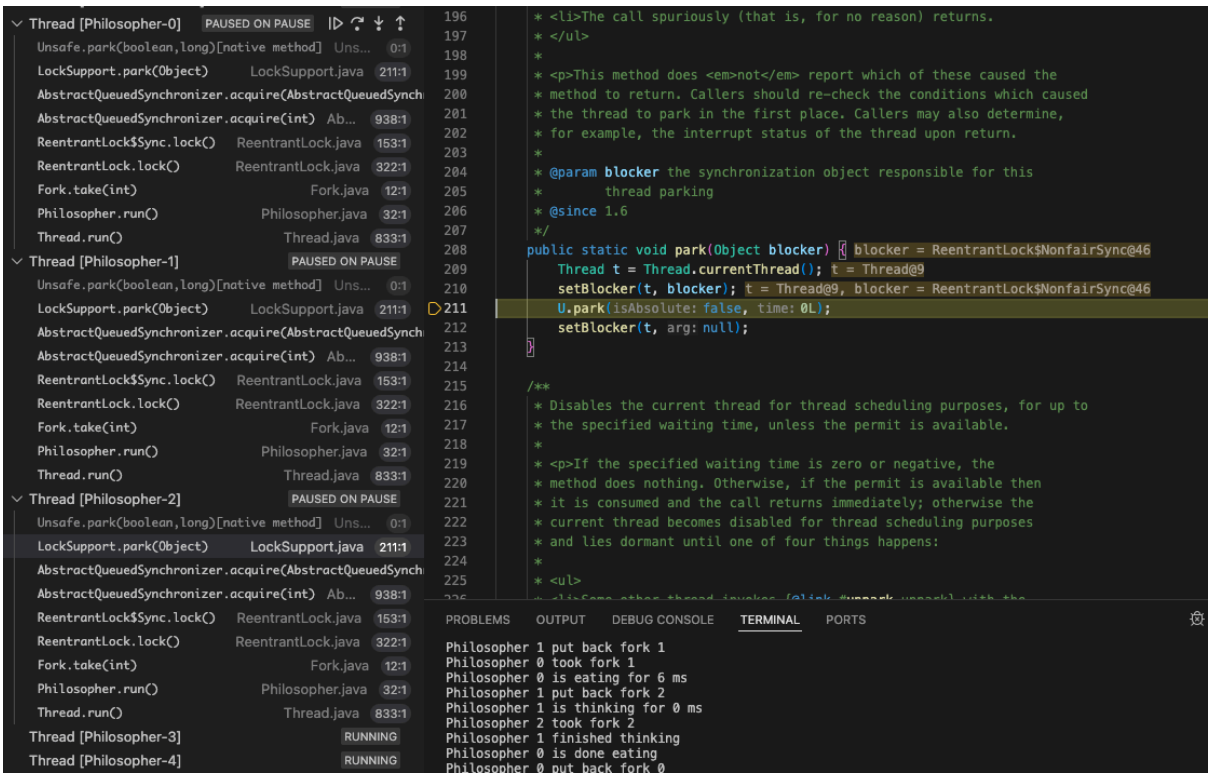
The program was run with 5 philosophers, a thinking time of 0 ms, and an eating time of 200 ms. The deadlock occurred almost immediately due to the minimal thinking time, giving all philosophers nearly simultaneous access to their left forks.

Debugger Analysis

After the program froze, execution was paused using the VSCode debugger. Inspecting the Call Stack panel revealed that all philosopher threads were stuck at the same point in the call stack:

```
LockSupport.park(Object)
ReentrantLock.lock()
Fork.take(int)
Philosopher.run()
```

The threads were blocked inside LockSupport.park(false, 0L), a native call that suspends a thread indefinitely until another thread releases the lock. Since every philosopher was parked waiting for their right fork, and no philosopher ever reached the putBack() call, no lock was ever released.



Root Cause

All philosophers follow the same fork acquisition order: left fork first, then right fork. In the worst case, all philosophers pick up their left fork simultaneously. Each then waits for the right fork, which is the left fork of their neighbour. This creates a circular dependency:

```
p0 holds f0, waits for f1
p1 holds f1, waits for f2
p2 holds f2, waits for f3
...
p(n-1) holds f(n-1), waits for f0
```

No philosopher can proceed, and the program is permanently frozen. Even pressing ENTER to trigger the cooperative shutdown has no effect, because the running flag is only checked at the top of the while loop, which the philosopher threads never reach.

Influence of Input Values on Deadlock Runtime

The following scenarios were tested with 4 philosophers unless stated otherwise.

Scenario 1: Low thinking time, low eating time (e.g. 0ms / 50ms)

Deadlock occurs almost instantly. With zero thinking time all philosophers rush for their left fork simultaneously at the start. Forks are held only briefly, but the initial simultaneous grab is enough to form the circular wait before anyone finishes eating. The deadlock forms in the very first cycle.

Scenario 2: High thinking time, low eating time (e.g. 500ms / 50ms)

Deadlock still occurs but takes longer. Philosophers spend most of their time thinking, so they reach for forks at different times, naturally reducing synchronisation. However, given enough time, random scheduling will eventually align them closely enough to form the circular wait. It is a matter of when, not if.

Scenario 3: 8 philosophers, 100ms thinking, 100ms eating -> appears deadlock-free

With 8 philosophers and balanced times the deadlock does not appear to occur within a reasonable observation window. The larger number of philosophers actually works against the deadlock here, for a circular wait to form, all 8 philosophers must grab their left fork before any single one of them manages to also grab their right fork. With random sleep times, the probability of all 8 aligning simultaneously is very low. Some philosopher will

almost always finish eating and release forks before the full circle closes. The deadlock is still theoretically possible but statistically unlikely in short runs.

Key Takeaway

The runtime until deadlock is determined by how likely it is for all philosophers to simultaneously hold exactly one fork. Low thinking time maximises this chance, high eating time sustains the dangerous state longer, and more philosophers require a larger, and therefore less likely, simultaneous alignment.

Subtask 2: Deadlock prevention

ˆ

What are the necessary conditions for deadlocks (discussed in the lecture)? ˆ

A deadlock can only occur if all four of the following conditions (Coffman conditions) hold simultaneously:

1. Mutual Exclusion: a resource can only be held by one thread at a time. Other threads must wait until it is released.
2. Hold and Wait: a thread is holding at least one resource while waiting to acquire additional resources held by other threads.
3. No Preemption: a resource cannot be forcibly taken away from a thread. It can only be released voluntarily by the thread holding it.
4. Circular Wait: a circular chain of threads exists where each thread is waiting for a resource held by the next thread in the chain.

Why does the initial solution lead to a deadlock (by looking at the deadlock conditions)?

All four conditions are present in the naive implementation:

1. Mutual Exclusion: each fork is protected by a ReentrantLock. Only one philosopher can hold a fork at a time.
2. Hold and Wait: after taking the left fork (fork[i]), the philosopher does not release it while blocking on the right fork (fork[(i+1) % n]).
3. No Preemption: lock.lock() blocks passively. No other thread or the system can take the fork away from a philosopher who holds it.
4. Circular Wait: philosopher p0 waits for p1's fork, p1 waits for p2's fork, ..., p(n-1) waits for p0's fork. A full circle forms across all philosophers.

Since all four conditions are satisfied simultaneously, a deadlock is inevitable.

Adrian Hansen, Marko Misic

Prevent the deadlock by removing Circular Wait condition:
Switch the order in which philosophers take the fork by using the following scheme: Odd philosophers start with the left fork, while even philosophers start with the right hand.

Does this strategy resolve the deadlock and why?

A: Yes it does.
The deadlock occurred because all philosophers followed the same rule: grab the left fork first, then the right. In the worst case, all philosophers grabbed their left fork simultaneously but now nobody could grab their right fork, because it was already held by their neighbour. Since nobody was releasing their left fork either, a full circular wait formed and the program froze.
By reversing the fork order for even philosophers, the only situation that can now occur is two neighbouring philosophers competing for the same fork at the same time. One of them wins, grabs both forks, eats, and releases them again. The other one simply waits briefly and then proceeds. The circular wait can never fully form so no deadlock.

Measure the overall time, philosophers spend for eating. How does it compare to the overall time execution time of the dinner.

The total time philosophers spent eating significantly exceeds the wall-clock duration of the dinner. This is because multiple philosophers eat concurrently, their eating times overlap in real time but are summed individually. The parallelism factor reflects how many philosophers were eating simultaneously on average.

```
--- Dinner Statistics ---
Wall-clock time:      7851 ms
Total eating time:    19109 ms
Parallelism factor:   2,43x
```

Can you think of other techniques for deadlock prevention?

1. Resource Ordering
Assign a global number to every fork and require all philosophers to always acquire forks in ascending order by number, regardless of which side they are on. This removes circular wait because no philosopher will ever hold a higher-numbered fork and wait for a lower-numbered one the chain can never loop back on itself.

Philosopher	Left fork	Right fork	Picks up first	Picks up second
p0	f0	f1	f0	f1
p1	f1	f2	f1	f2
p2	f2	f3	f2	f3
p3	f3	f0	**f0**	f3

p3 would grab f3 (his left) first, but because f0<f3 he goes for f0 which breaks the circle.

2. Timeout with Backoff
Instead of blocking indefinitely on a fork, a philosopher tries to acquire it with a timeout (tryLock). If he cannot grab the second fork within the timeout, he releases the first fork and tries again after a short random delay. This breaks the "hold and wait" condition a philosopher never holds one fork while waiting forever for another.

Make sure that your final solution compiles, runs without race conditions or deadlock and performs a proper cooperative shutdown, and uses as little as possible but as many locks as needed.

Using ReentrantLock with fair lock flag to ensure FIFO ordering and prevent starvation among philosophers and busy waiting

```
private final ReentrantLock lock = new ReentrantLock(true);
```

Using try and finally blocks so that an exception inside the philosopher eat method does not cause it to keep the fork forever. It always gives the fork back, also if there is an exception.

```
try {
  while (running) {
    think();
    Fork first = (index % 2 != 0) ? leftFork : rightFork;
    Fork second = (index % 2 != 0) ? rightFork : leftFork;

    first.take(index);
    try {
      second.take(index);
      try {
        eat();
      } finally {
        second.putBack(index);
      }
    } finally {
      first.putBack(index);
    }
  }
} catch (Exception e) {
  System.out.println("Philosopher " + index + " encountered an error: " + e.getMessage());
} finally {
  System.out.println("Philosopher " + index + " cleanup and leaving table.");
}
```